Guía de exposición — Christian Méndez

Curso: Desarrollo Web · **Proyecto:** Vivent · **Ciclo:** octubre 2026

Documento de apoyo individual. No sustituye al `DOCUMENTO_PROYECTO.md`; lo resume desde tu rol.

---

0. Tu rol en una tabla

Elemento	Lo que te corresponde
Módulos (2)	Catálogo · Inventario
Proceso integrado (1)	P1 — Compra confirmada → capa FIFO → kardex → existencia actualizada
Reportes (3)	Existencias por sucursal · Kardex por producto · Productos con stock bajo
API externa (1)	**Photon** (geocodificación de direcciones)
Transversales	Autenticación y permisos · Auditoría
Carpetas de Postman	`01 · Autenticación`, `03 · Catálogo`, `05 · Inventario`, `11 · Integraciones externas` (Photon), `13 · Auditoría`

> Tu proceso (P1) usa la compra que construyó Wilfero: tú demuestras **el efecto** (capa FIFO, kardex, saldo), no la pantalla de compras. Coordinen quién confirma la compra en la demo en vivo para no repetir.

---

1. Guion de 12 minutos

Min	Qué muestras	Qué dices
0–1	`http://localhost:3000` → login `admin` / `ChangeMe123!`	"El login resuelve el tenant por subdominio y devuelve un JWT con los permisos del rol; el backend valida empresa, sucursal, suscripción y usuario antes de emitirlo."
1–3	Catálogo → Productos → crear un producto con unidad `can` y una presentación con código de barras	"Todo producto tiene unidad base; las presentaciones solo convierten a esa unidad (1, 6, 24). El inventario siempre se mueve en unidad base."
3–5	Inventario → Existencias · pestaña Capas FIFO	"Cada compra confirmada crea una capa con su costo. La venta consume primero la capa más antigua, por eso el costo del kardex es exacto."
5–7	Proceso P1: abrir la compra `ACAD-PUR-001` (ya confirmada) y volver a Inventario	"Antes de confirmar no existía stock; al confirmar se creó la capa, subió la existencia y se escribió el movimiento de kardex `PURCHASE_IN`."
7–9	Reportes: Existencias → filtro por sucursal → **Exportar CSV** → **Imprimir**	"Los nueve reportes filtran en el servidor y exportan exactamente lo filtrado, con BOM UTF-8 para que Excel abra bien los acentos."
9–10	Reportes → Kardex por producto (filtro por producto y rango de fechas)	"El kardex es la historia completa: entradas, salidas y saldo después de cada movimiento."
10–11	Reportes → Productos (pestaña Stock bajo)	"Compara contra el stock mínimo configurado por producto; es el insumo para reponer."
11–12	Postman: `11 · Integraciones externas` → Photon	"Integración real con Photon: devuelve latitud, longitud y etiqueta; con caché de 10 minutos y timeout de 4 segundos."

---

2. Postman: qué ejecutar y en qué orden

Abre `postman/Vivent.postman_collection.json` con el environment **Vivent Local** y ejecuta el login primero.

#	Request exacto	Resultado esperado	Qué demuestras
1	`01 · Autenticación → Login (guarda tokens)`	`200`, guarda `accessToken`	Emisión de JWT
2	`01 · Autenticación → Sesión actual (/auth/me)`	`200` con rol y permisos	El token se valida en cada request
3	`01 · Autenticación → Refresh token (renueva y rota)`	`200` con par nuevo	Renovación sin volver a pedir clave
4	`03 · Catálogo → Categorías → Crear categoría`	`201/200`	CRUD con validación
5	`03 · Catálogo → Productos → Crear producto`	`200` con `id`	Unidad base obligatoria
6	`03 · Catálogo → Presentaciones → Crear presentación`	`200`	Factor de conversión y código de barras
7	`03 · Catálogo → Búsqueda por código de barras → Buscar en el catálogo propio`	`200`	Resolución de presentación para el POS
8	`05 · Inventario → Capas FIFO`	lista con `quantityRemaining`	Costeo por capas
9	`05 · Inventario → Kardex`	movimientos con `quantityIn/Out`	Trazabilidad
10	`05 · Inventario → Entrada manual de stock`	`200`	Ajuste controlado con permiso
11	`11 · Integraciones externas → Photon: geocodificar dirección`	`found: true`	API externa
12	`11 · Integraciones externas → Photon: sin dirección (error)`	`found: false`	Manejo de error
13	`13 · Auditoría → Bitácora`	movimientos con usuario y acción	Auditoría

Si algo falla en vivo: ejecuta de nuevo el login; si el token expiró (30 min) el refresh lo renueva. Si el puerto 8080 no responde, revisa `docker ps` y `docker logs vivent-academico-backend`.

---

3. Código: dónde apuntar

Tema	Archivo	Punto exacto
Login multiempresa	`backend/.../auth/service/AuthService.java`	`login()` → `loadTenant()`, `validateCompany()`, `validateSubscription()`, `validateUserAccess()`
Emisión del JWT	`backend/.../security/JwtService.java`	Firma y claims (`companyId`, `branchId`, `permissions`)
Filtro de cada request	`backend/.../security/JwtAuthenticationFilter.java`	Valida token, usuario activo y `auth_version`
Permisos efectivos	`backend/.../permissions/service/PermissionResolutionService.java`	`resolvePermissions()` = rol + overrides (DENY antes que GRANT)
Catálogo: SKU único	`backend/.../products/service/ProductService.java`	`createProduct()` → `validateSkuAvailable()`
Presentaciones	`backend/.../products/service/ProductPresentationService.java`	`createPresentation()`, `lookupBarcode()`
FIFO (lo más importante)	`backend/.../inventory/service/InventoryService.java`	`receiveStock()` crea la capa (líneas ~346–360) y **`issueStock()` consume las capas más antiguas** (líneas ~580–604)
Orden de capas	`backend/.../inventory/repository/InventoryLayerRepository.java`	`findOpenLayersForUpdate()` (orden por `receivedAt`)

Tema	Archivo	Punto exacto
Kardex	`backend/.../inventory/service/InventoryService.java`	`getKardex()`, `listMovements()`
Auditoría	`backend/.../audit/service/AuditLogService.java`	`log(...)` se llama al confirmar, anular, aprobar, etc.
Frontend de inventario	`frontend/src/features/inventory/`	`components/inventory-screen.tsx`, `api/inventory-api.ts`
Reportes (pantalla)	`frontend/src/app/reports/inventory/page.tsx` · `kardex` · `products`	Enlazan a `features/reports/components/reports-page.tsx`
Exportar / imprimir	`frontend/src/components/shared/report-actions.tsx`	Botones **Exportar CSV** e **Imprimir**

---

4. Tus tres reportes

Todos están en `Reportes` del menú lateral.

Reporte	Ruta	Endpoint que consume	Filtros que debes mover en la demo
Existencias por sucursal	`/reports/inventory`	`GET /api/v1/inventory/balances`	Sucursal, categoría, producto, texto
Kardex por producto	`/reports/kardex`	`GET /api/v1/inventory/kardex`	Producto, sucursal, tipo de movimiento, fechas
Productos con stock bajo	`/reports/products`	`GET /api/v1/reports/products`	Sucursal, categoría, rango de fechas, pestaña «Stock bajo»

Guion de 60 segundos para cada uno: aplica **un** filtro → muestra que cambian las filas → pulsa **Exportar CSV** (di: "el archivo contiene solo lo filtrado") → pulsa **Imprimir** y muestra la vista previa (di: "el CSS de impresión oculta el menú y deja solo el reporte").

> El peso de la rúbrica está en **filtros + impresión/exportación**, no en la cantidad de columnas. Muestra siempre las dos acciones.

---

5. Tu API externa: Photon

Dato	Valor
Endpoint	`GET /api/v1/integrations/geocode?q=Parque%20Central%20Cuilapa`
Código	`backend/.../integrations/geocoding/controller/GeocodingController.java` + `service/GeocodingService.java`
Respuesta de éxito	`{ query, source: "PHOTON", found: true, latitude, longitude, label }`
Respuesta de error	`{ found: false, message }` (sin excepción para el usuario)
Protecciones	Caché en memoria 10 min · timeout 4 s · `permission: settings.view`

Qué decir: "Photon es un geocodificador libre. Lo uso para normalizar la dirección de la sucursal; si el servicio externo falla, la API no se cae: devuelve `found=false` con mensaje y el sistema sigue funcionando."

Caso de error para mostrar: `?q=` vacío → `400` de validación; o una dirección inexistente → `found: false`.

---

6. Rúbrica: cómo la cumples tú

Criterio de la rúbrica	Cómo se cumple	Evidencia que muestras
2 módulos funcionales	Catálogo (unidades, categorías, productos, presentaciones, combos) e Inventario (existencias, capas, kardex, entradas/salidas)	Recorrido en vivo + `03` / `05` de Postman
1 proceso integrado	P1: compra confirmada → capa FIFO → kardex → saldo	Compra `ACAD-PUR-001` + capa y movimiento
3 reportes con filtros	Existencias, Kardex, Productos (stock bajo)	Filtro aplicado en cada uno
Impresión / exportación	Botones de `report-actions.tsx` en los tres	CSV descargado + vista previa de impresión
1 API externa	Photon con éxito y error	Requests `11 · Integraciones externas`
≥5 registros por catálogo	Seed `S20261026000001__seed_academic_demo_data.sql` : 8 unidades, 5 categorías, 5+ proveedores, 5+ clientes, 10+ presentaciones	Tabla de catálogos con el total visible
JWT access + refresh	Tu parte: emisión y validación (`JwtService` , `JwtAuthenticationFilter` , `PermissionResolutionService`)	`01 · Autenticación` + `/auth/me`
CRUD en Postman	31 requests de catálogo + 14 de inventario	Run de la carpeta
Documentación	`DOCUMENTO_PROYECTO.md` §5.1–5.2, §7.2, §11	PDF abierto en la sección
Commits propios	Pendiente: debes subir tu propia contribución	`git log --author="Christian"`

---

7. Preguntas probables

1. **¿Por qué el inventario se guarda en unidad base y no en cajas?** Porque una venta puede ser 1 lata, 1 six pack o 1 caja; si guardáramos cajas, media caja rompe el saldo. La conversión ocurre en la presentación.
2. **¿Cómo garantizas el costo real?** Con capas FIFO: cada compra entra con su costo y `issueStock()` consume de la más antigua a la más nueva, dejando el costo exacto en `sale_details.fifo_cost_total` .
3. **¿Qué pasa si no hay stock?** `issueStock()` lanza `INSUFFICIENT_STOCK` antes de tocar la base; la venta no se confirma.
4. **¿Cómo evitas que una empresa vea otra?** Todas las consultas filtran por `company_id` del token y, si el usuario no es COMPANY_ADMIN, además por su sucursal (`appendBranchScope`).
5. **¿Los permisos se validan en el frontend?** El frontend solo oculta botones; la decisión real está en el backend con `@PreAuthorize` sobre permisos `módulo.acción` .
6. **¿Las bajas borran registros?** No: son lógicas (`deleted_at` / `is_active`); por eso el kardex y la auditoría nunca pierden historia.
7. **¿Qué pasa si Photon no responde?** Timeout de 4 s, se devuelve `found=false` y la pantalla avisa; no se bloquea el guardado de la sucursal.

---

8. Checklist 10 minutos antes

- [ ] `docker ps` muestra `vivent-academico-db` , `-backend` y `-frontend` arriba.
- [ ] `python database/bootstrap_demo.py` corrido (compras, ventas, traslado, ajuste, cierre).
- [ ] Login `admin` funciona y el menú muestra Catálogo, Inventario y Reportes.
- [ ] Postman importado con environment **Vivent Local** y login ejecutado.
- [ ] `documento-proyecto.pdf` abierto en la sección 5.1 (catálogo) y 7.2 (inventario).
- [ ] Pestaña del navegador con `/reports/kardex` ya cargada (por si el filtro tarda).
- [ ] Sabes dónde está la compra `ACAD-PUR-001` para mostrar el efecto FIFO.